Assignment 4: Heap Exploitation

Secure Systems Engineering (CS6570)
Indian Institute of Technology Madras

Overview

You are given **VaultSrv** binary implementing a vault management service, compiled against **glibc 2.23**. The assignment has three progressive stages, each having a different heap exploitation concept. All three stages run against the same binary. You restart the binary for each stage.

Stage	Name	Concept	Marks
1	Bin Anatomy	Fastbin LIFO ordering & heap address leak	20
2	Prev Size Lies	Off-by-one null byte → chunk overlap	40
3	Unlink Undone	Unsafe unlink → arbitrary write primitive	40

Upon successful exploitation, each stage prints a flag from a local file.

Files Provided

- `vaultsrv` - the binary (ELF 64-bit, not stripped, symbols visible)
- `libc-2.23.so` - the exact glibc shared library
- `ld-2.23.so` - the matching dynamic linker
- `flag1.txt`, `flag2.txt`, `flag3.txt` - local test flags
- `Dockerfile` - Docker container
- `start.sh` - Docker environment launcher

```
Write and run your exploit
python3 exploit_stage1.py
```

Important:

- You **must** use the provided Docker environment (`start.sh`). The Dockerfile patches the binary to use glibc 2.23 automatically. If you run the binary outside Docker, it may use your system's glibc (2.31+), which has `tcache` and completely different heap behavior.

Stage Descriptions

Stage 1 - Bin Anatomy (20 marks)

Goal: Leak a specific heap address and submit it via `claim` (menu option 10, stage 1).

Hints: Which function frees memory? Does it properly clean up? Which function lets you read freed data?

Stage 2 - Prev Size Lies (40 marks)

Goal: Write the magic value `0xC0FFEE1337` into a specific vault's heap region via chunk overlap. Submit via `claim` (stage 2).

Hints: Which function writes bytes past the allocation? What allocation size makes those extra bytes land on the next chunk's size field?

Stage 3 - Unlink Undone (40 marks)

Goal: Overwrite a random 64-bit value stored in BSS (it cannot be guessed, it must be overwritten). Submit the value you wrote via `claim` (stage 3).

Hints: What BSS variable holds the random target? What other BSS state must you preserve during the write? You will need a libc leak (unsorted bin) to preserve a critical pointer.

Submission Requirements

Your submission will be evaluated by an **automated grading script**. This means:

Do NOT hardcode flag values. Your exploit must actually trigger the vulnerability and cause the binary to print the flag. Simply reading `flag1.txt` and printing its contents will score zero..

File Naming Convention

Submit a folder named with your roll number containing:

```
<roll_number>/
    exploit_stage1.py # Stage 1 exploit (pwntools)
    exploit_stage2.py # Stage 2 exploit (pwntools)
    exploit_stage3.py # Stage 3 exploit (pwntools)
    report.pdf # Detailed report for all stages
```

Exploit Script Requirements

Each exploit script must:

1. Be a standalone Python 3 script using pwntools
2. Run from the same directory as `vaultsrv`, `libc-2.23.so`, etc.
3. Launch the binary via `process('./vaultsrv')`
4. Produce the flag through the binary's own output (not by reading flag files directly)
5. Exit cleanly after printing the flag

Report Requirements

A single PDF (`report.pdf`) covering all stages you solved. For each stage:

- How you analyzed the binary (which functions, which globals)
- The vulnerability and why it exists
- Heap layout diagrams at each critical step
- The exploit chain and why it works
- Challenges faced and how you debugged them

Marks Distribution

Component	Marks	Evaluation
Stage 1: Bin Anatomy	20	10 auto-grader + 10 report
Stage 2: Prev Size Lies	40	20 auto-grader + 20 report
Stage 3: Unlink Undone	40	20 auto-grader + 20 report
Total	**100**	

Partial credit for correct analysis in the report, even without a working exploit.
All external resources used must be cited in your report.